

CORRECCIÓN

Revisión del modelo:

- Revisar la clave primaria de la tabla capítulos.

```
CREATE TABLE capitulos (  
  id_serie INT NOT NULL,  
  temporada INT NOT NULL,  
  numero INT NOT NULL,  
  duracion SMALLINT NOT NULL,  
  PRIMARY KEY (id_serie, temporada, numero),  
  FOREIGN KEY (id_serie)  
    REFERENCES series(id_contenido)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Hemos cambiado la clave primaria 'codigo_capitulo' por una union entre los diferentes campos ya existentes de la tabla, siendo estos 'id_serie', 'temporada' y 'numero', para generar claves primarias en función a los datos ya conocidos de estas entradas.

- Añadir columnas a la tabla serie para justificar su existencia.

```
CREATE TABLE series (  
  id_contenido INT NOT NULL,  
  numero_temporadas INT NOT NULL,  
  en_emision BOOLEAN NOT NULL,  
  PRIMARY KEY (id_contenido),  
  FOREIGN KEY (id_contenido)  
    REFERENCES contenido(id_contenido)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

Hemos añadido los campos 'numero_temporadas' y 'en_emision' a la tabla serie.

- Revisar la clave primaria de la tabla contrata, ¿no sería conveniente utilizar el código de trabajador?
No, ya que es una relación N:M, un trabajador puede trabajar en varias productoras, y una productora puede tener varios trabajadores. Nos interesa que la relación sea de esta manera.

- Al tipo de usuario, ¿se le podrían limitar los valores a incluir con un tipo de datos enum o un check?

```
CREATE TABLE usuario (
  DNI VARCHAR(15) NOT NULL,
  nombre VARCHAR(50) NOT NULL,
  apellidos VARCHAR(50) NOT NULL,
  correo VARCHAR(80) NOT NULL,
  telefono VARCHAR(20) NOT NULL,
  tipo ENUM('Básico', 'Estándar', 'Premium') NOT NULL,
  PRIMARY KEY (DNI)
) ENGINE=InnoDB;
```

Si, era más coherente cambiar el tipo a un enum con los valores 'Básico', 'Estándar', 'Premium'.

- ¿Podéis ajustar más el tipo de dato de la columna puntuación de reseña?

```
CREATE TABLE reseña (
  codigo_perfil INT NOT NULL,
  id_contenido INT NOT NULL,
  fecha DATE NOT NULL,
  puntuacion TINYINT NOT NULL,
  comentario TEXT,
  PRIMARY KEY (codigo_perfil, id_contenido),
  FOREIGN KEY (codigo_perfil)
    REFERENCES perfil(codigo_perfil)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  FOREIGN KEY (id_contenido)
    REFERENCES contenido(id_contenido)
    ON DELETE CASCADE
    ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Si, al ser un valor del 1 al 10, un TINYINT es más coherente que un INT normal.

- Revisar el tipo de datos utilizado para almacenar los premios.

```
CREATE TABLE productora (
  codigo_productora INT NOT NULL,
  nombre_productora VARCHAR(100) NOT NULL,
  premios VARCHAR(255),
  PRIMARY KEY (codigo_productora)
) ENGINE=InnoDB;
```

El tipo TEXT era excesivo, mejor dejarlo en VARCHAR(255).

Revisión Consultas:

- La consulta 5 devuelve que sólo hay un contenido de género "Terror" y la consulta 6 que tiene 2. Revisar.

```

31  -- -----
32  -- 5. GROUP BY Simple
33  -- Mostrar cuanto contenido hay de cada género.
34  -- -----
35  • SELECT genero, COUNT(*) FROM contenido GROUP BY 1;
36

```

#	genero	COUNT(*)
1	Ciencia Ficción	1
2	Terror	2
3	Aventura	1
4	Fantasía	1

- Revisar si la consulta 14 devuelve lo que se indica en el enunciado para todos los casos posibles.

```

95  -- -----
96  -- 14: RIGHT JOIN
97  -- Mostrar todos los perfiles, junto con las reseñas que han hecho, mostrar tambien los perfiles sin reseñas.
98  -- -----
99  • select r.comentario, r.id_contenido, p.nick
100  from reseña r
101  right join perfil p
102  on r.codigo_perfil = p.codigo_perfil;
103

```

#	comentario	id_contenido	nick
1	Buena pero algo lenta	103	JuanKids
2	Excelente película	100	JuanMain
3	Muy entretenida	101	AnaPerfil
4	Un clásico	102	CarlosPerfil
5	NULL	NULL	JoseMartin

- Revisar si la consulta 15 retorna lo que se indica en el enunciado.

```

104  -- -----
105  -- 15.SELF JOIN (tabla consigo misma)
106  -- Consultar los jefes cuyo DNI es vacío.
107  -- -----
108  • SELECT * FROM trabajadores t
109  INNER JOIN trabajadores j
110  ON t.DNI_jefe = j.DNI
111  WHERE j.DNI_jefe IS NULL;
112

```

#	DNI	nombre	apellidos	tipo_trabajador	DNI_jefe	DNI	nombre	apellidos	tipo_trabajador	DNI_jefe
1	90000002B	Miguel	Santos	Productor	90000001A	90000001A	Laura	Gómez	Directora	NULL

- Complicar más la consulta 19

```

140  -- -----
141  -- 19.JOIN con Múltiples Condiciones
142  -- Mostrar el perfil del usuario con DNI 12345678A, cuyo nombre de usuario sea JuanKids, o los usuarios cuyo apellido sea Martín.
143  -- -----
144  • SELECT * FROM perfil p
145    INNER JOIN usuario u
146    ON u.DNI = p.DNI_usuario
147   WHERE u.DNI = '12345678A'
148    AND p.nick = 'JuanKids'
149    OR u.apellidos = 'Martín'
150    AND telefono = '666666666';
151

```

#	codigo_perfil	nick	DNI_usuario	DNI	nombre	apellidos	correo	telefono	tipo
1		JuanKids	12345678A	12345678A	Juan	Pérez	juan@gmail.com	600111222	Premium
2	5	JoseMartín	12345679B	12345679B	Jose	Martín	jose@gmail.com	666666666	Básico

- Consulta 20 marcar las que son de cada tipo.

```

152  -- -----
153  -- 20: Unión de Resultados (UNION) (stephano)
154  -- Muestra los nombres de películas y series en una sola lista, indicando series con S y películas con P.
155  -- -----
156  • SELECT nombre_contenido, 'P'
157    FROM contenido
158   WHERE tipo = 'Película'
159  UNION
160  SELECT nombre_contenido, 'S'
161    FROM contenido
162   WHERE tipo = 'Serie';
163

```

#	nombre_contenido	P
1	Inception	P
2	Jurassic Park	P
3	The Frog	P
4	Stranger Things	S
5	The Witcher	S

- Complicar más la consulta 21

```

164  -- -----
165  -- 21.LEFT JOIN CON filtro complejo
166  -- pide el nombre del contenido y su puntuacion
167  -- de la tabla reseña que su puntuacion sea
168  -- mayor que 8 o nulo.
169  -- -----
170  • SELECT c.nombre_contenido, r.puntuacion
171    FROM contenido c
172   LEFT JOIN reseña r
173   ON c.id_contenido = r.id_contenido
174   WHERE (r.puntuacion > 8 AND c.tipo = 'Película') OR (r.puntuacion IS NULL AND c.nombre_contenido = 'The Frog');
175

```

#	nombre_contenido	puntuacion
1	Inception	9
2	Jurassic Park	10
3	The Frog	NULL

- Revisar consulta 22. Pueden existir muchos "Juan" en la tabla usuarios.

```

176  -- -----
177  -- 22: Subconsulta en WHERE
178  -- Mostrar el perfil de todos los usuarios con nombre Juan.
179  -- -----
180  • select * from perfil
181  where DNI_usuario IN (select DNI from usuario
182                        where nombre = 'Juan');
183

```

#	codigo_perfil	nick	DNI_usuario
1	1	JuanKids	12345678A
2	2	JuanMain	12345678A
3	NULL	NULL	NULL

- Incorrecta la consulta 27

```

227  -- -----
228  -- 27.Subconsulta en FROM (Tabla Derivada)
229  -- Mostrar los contratos de la productora con id = 1.
230  -- -----
231  • SELECT *
232  FROM contrata c
233  INNER JOIN (
234      SELECT *
235      FROM productora
236      WHERE codigo_productora = 1
237  ) p
238  ON c.codigo_productora = p.codigo_productora;
239

```

#	codigo_productora	DNI_trabajador	codigo_productora	nombre_productora	premios
1	1	90000001A	1	Warner Bros	Oscar 2020
2	1	90000002B	1	Warner Bros	Oscar 2020

- Revisar definición de la consulta 29.

```

252  -- -----
253  -- 29.SUBCONSULTAS con ANY/SOME.
254  -- Mostrar cualquier contenido que se haya publicado en la fecha 2010-07-16
255  -- -----
256  • SELECT nombre_contenido, fecha_salida FROM contenido
257  WHERE fecha_salida = ANY (
258      SELECT fecha_publicacion FROM peliculas
259      WHERE fecha_publicacion = '2010-07-16');
260

```

#	nombre_contenido	fecha_salida
1	Inception	2010-07-16